

Tamper-Evident Precinct And Batch Hashing

Gio Della-Libera

Draft — June 28, 2026

Abstract

Public election-count evidence is fragile unless the bytes being verified are named, hashed, and replayed. This paper describes RCOUNT’s tamper-evidence layer: source hashes for raw public inputs, package hashes for normalized records, batch manifests for smaller accounting units, and transcripts that separate hash failures from arithmetic failures. Hashing does not certify an election or prove tabulator correctness. It makes later changes visible: if a source file changes after publication, a verifier can detect the mismatch even when summary arithmetic still passes.

1 Introduction

A public count package has two different integrity problems. First, the package must preserve the meaning of public election records: contests, reporting units, batches, statuses, residual counts, and source references. Second, it must make later byte-level changes visible. A normalized summary can still add correctly after the raw source file has been edited, replaced, truncated, or omitted.

RCOUNT therefore separates arithmetic checks from hash checks. A verifier may report that local contest sums and jurisdiction totals pass while the source hash check fails. That is not a contradiction. It means the normalized records reconcile, but the raw evidence no longer matches the package’s declared source index.

The discipline is tamper evidence, not omniscience. Hashes can show that bytes changed. They cannot show that the original bytes were lawful, complete, accurately exported, free of malware, or sufficient for certification.

2 Hash Layers

RCOUNT uses separate hash roles so that verifier transcripts can name the layer that failed.

Layer	Object	Purpose
Source hash	Raw public input file	Detects byte changes in the evidence an adapter read.
Record hash	Normalized record	Supports stable hashes over canonicalized rows.
Event hash	Status or lineage event	Binds typed public events to canonical records.
Package hash	RCOUNT package content	Names the normalized claim set as a whole.
Proof hash	Public proof object	Names inclusion-proof material without exposing choices.

The current crates use domain-separated SHA-256 prefixes for these roles. The domain separation matters because a hash of a source file should not be confused with a hash of a package, event, or proof. Each role answers a different question.

Prefix	Hash role
RCOUNT_SOURCE_V1	Raw source file bytes.
RCOUNT_RECORD_V1	Normalized canonical record.
RCOUNT_FILE_V1	Canonical normalized file projection.
RCOUNT_PACKAGE_V1	Package content hash.
RCOUNT_EVENT_V1	Status or canvass event.
RCOUNT_PROOF_V1	Public proof object.

The suffix is part of the contract. Future RCOUNT versions may migrate hash algorithms or canonical projections, but the prefix and algorithm must be recorded so older packages remain replayable.

The **summary-basic** package currently records a package content hash and simple package counts:

```
{
  "package_content_hash": "sha256:2e8876...",
  "contest_count": 1,
  "reporting_unit_count": 3,
  "batch_count": 0,
  "summary_count": 3
}
```

That digest names the normalized package. It does not replace the source index: an auditor still needs to know which raw files were read and whether those raw file hashes match.

The normalized package hash covers the package's contests, reporting units, batches, lineage records, inclusion-proof records, summaries, and status events after canonical JSON projection. Volatile metadata such as local file mtimes, temporary paths, pretty-printing, and transcript generation time is not part of that normalized package content.

3 Source Index Contract

Source hashes bind raw bytes. Normalized hashes bind canonical records. The source index is the bridge between those layers.

Field	Contract
<code>source_id</code>	Stable package identifier for the source artifact.
<code>path</code>	Package-relative path under <code>sources/</code> ; absolute paths and parent-directory escapes are invalid.
<code>sha256</code>	Domain-separated hash over the raw source bytes.

The current synthetic source index is intentionally small:

```
{
  "source_id": "source:synthetic-summary-export",
  "path": "sources/synthetic-summary-export.json",
  "sha256": "sha256:f1fd7040..."
}
```

Real adapters will need more metadata: source format, export system, parser version, public-record citation, custody note, redaction status, and parser diagnostics. That richer adapter contract belongs to V.9. V.3 only requires the baseline replay rule: the package names the source path, hashes the raw bytes, and rejects empty or missing source evidence.

4 Batch Manifests

Batch manifests make the hashing and accounting surface smaller than the whole election. A package can describe election-day batches, mail batches, provisional batches, or central-count batches and bind each manifest to source references.

In `mail-batch-added`, the batch ledger contains:

Batch	Accepted	Counted	Rejected	Source reference
P-001 election day	70	70	0	<code>synthetic-summary-export</code>
P-001 late mail	10	10	0	<code>synthetic-summary-export</code>
P-002 election day	60	60	0	<code>synthetic-summary-export</code>

The manifest gives RCOUNT two kinds of evidence. The arithmetic evidence checks that accepted ballots equal counted plus rejected ballots. The source evidence checks that the public input bytes referenced by the batch still match the source index. A late mail batch is therefore not treated as suspicious because it is late; it is auditable because it is declared, reconciled, and source-bound.

Batch ids are package-scoped. A batch source reference says which declared source artifact supports the manifest row. It does not prove that every ballot in the batch was interpreted correctly, that every rejection reason is complete, or that a CVR adapter preserved every field. Ballot manifest verification is V.5, and CVR-to-summary reconciliation is V.6.

5 Negative Fixtures

The hash layer earns its keep when adjacent checks disagree. Two negative fixtures show different failures:

Fixture	Failure	Interpretation
<code>tampered-source</code>	<code>source_hash_match</code> fails after raw source bytes are edited.	Arithmetic can still pass, but the evidence file no longer matches the declared hash.
<code>missing-source-hash</code>	<code>source_hash_match</code> fails because the source index is empty.	The package omits the raw source-hash evidence needed for replay.

In both cases, local contest sums and jurisdiction totals can pass. That is the point: hash failures are not arithmetic failures. A good verifier transcript keeps them separate so a public reader can ask the right follow-up question.

Fixture	Expected result	Carrying checks
<code>summary-basic</code>	pass	<code>contest_selection_sum</code> , <code>jurisdiction_contest_total</code> , <code>source_hash_match</code> .
<code>mail-batch-added</code>	pass	<code>accepted_ballots</code> , <code>batch_summary_total</code> , <code>source_hash_match</code> .
<code>tampered-source</code>	fail	<code>source_hash_match</code> reports a declared/computed hash mismatch.
<code>missing-source-hash</code>	fail	<code>source_hash_match</code> reports an empty source index.

6 Verifier Traceability

The fixtures can be regenerated and verified from the crate examples:

```
cargo run -p rcount-io --example summary_basic_package
cargo run -p rcount-io --example mail_batch_added_package
cargo run -p rcount-cli -- verify \
  docs/examples/rcount-golden-packages/summary-basic
cargo run -p rcount-cli -- verify \
  docs/examples/rcount-golden-packages/mail-batch-added
cargo run -p rcount-cli -- verify \
  docs/examples/rcount-golden-packages/tampered-source
cargo run -p rcount-cli -- verify \
  docs/examples/rcount-golden-packages/missing-source-hash
```

A compact excerpt from `tampered-source` shows the desired attribution:

```
{
  "status": "fail",
  "checks": [
```

```

    {"equation_id": "contest_selection_sum", "status": "pass"},
    {"equation_id": "jurisdiction_contest_total", "status": "pass"},
    {
      "equation_id": "source_hash_match",
      "status": "fail",
      "error": "source hash mismatch for source:synthetic-summary-export"
    }
  ]
}

```

The important property is not merely that the package fails. It fails at the source-evidence layer while preserving the fact that arithmetic checks passed.

The `missing-source-hash` fixture fails at the same layer for a different reason:

```

{
  "status": "fail",
  "checks": [
    {
      "equation_id": "source_hash_match",
      "status": "fail",
      "error": "source index is empty"
    }
  ]
}

```

The crate-level regression surface is split between `rcount-io`, which checks source-index rules, and `rcount-audit/rcount-cli`, which emit public verification transcripts.

7 Boundaries

Hashing is not certification. A passing RCOUNT hash check says that declared source bytes still match their recorded hashes. It does not say that an election official certified the result, that the source export was complete, or that a court would accept the record.

Hashing is also not encryption. If a source file contains private data, hashing the file does not make the data safe to publish. RCOUNT packages should publish only receipt-safe public evidence, and private support files should be handled under the jurisdiction's custody rules.

Finally, tamper evidence is not malware resistance or end-to-end cryptographic voting. RCOUNT can detect that a published source file changed after hashing. It cannot prove that a tabulator, ballot-marking device, or export pipeline was correct before that source file was produced.

Concern	RCOUNT hash layer can show	It does not show
Source byte change	Declared raw bytes no longer match.	Who changed them or why.
Source omission	Source index is empty or file is missing.	Whether another office record exists elsewhere.
Package mutation	Normalized content hash changed.	Whether the old or new content is legally correct.
Parser substitution	Different normalized records can be detected by package hash.	That the parser captured every source nuance.
Private data	Nothing; hashes are not redaction.	That sensitive files are safe to publish.
Malware or tabulator error	Nothing directly.	Device or export-pipeline correctness.
Certification	Evidence consistency.	Legal certification or court acceptance.

For public-record workflows, a certification artifact can itself be a hashed source. That still separates evidence integrity from the legal act of certification. Detached signatures, source-custody attestations, and superseding or amended source packages are future work.

8 Conclusion

Tamper-evident hashing gives RCOUNT a public memory. Source hashes bind raw inputs, package hashes bind normalized claim sets, and batch manifests bind smaller accounting surfaces to declared evidence. The result is not automatic certification. It is a transcript that can say exactly where a package still replays and where the evidence bytes no longer match.

That distinction supports the rest of the V track. Canvass arithmetic, precinct lineage, ballot manifests, CVR reconciliation, and district aggregation all need hash-bound evidence if public verification is going to be repeatable.

References

- Gio Della-Libera. Canvass arithmetic: From unofficial returns to certified totals, 2026a. Track V technical report.
- Gio Della-Libera. Precinct lineage across elections, 2026b. Track V technical report.
- Gio Della-Libera. Rcount overview: Reproducible election count packages, 2026c. Track V technical report.
- Gio Della-Libera. Rcount substrate specification, 2026d. Project specification.